How the Kernel Captures the IP Address

From hrtimer fire to ring buffer — step by step

Linux 5.10-rc1 | x86-64 | perf_event_open NR_298

1. The One Key Insight

The CPU **HARDWARE** takes it. The kernel only **READS** what hardware already saved.

HARDWARE does	Reads rip register. Pushes it to kernel stack. BEFORE any kernel code runs. One atomic operation. No software instruction involved.
SOFTWARE does	Completes the pt_regs struct around rip. Reads regs->ip from memory. Writes it to ring buffer.
pt_regs->ip	The saved rip value on kernel stack. Put there by hardware. Never changes once saved. Kernel reads it from memory, not from CPU register.
rip REGISTER	Changes every nanosecond as kernel code runs. By the time kernel reads pt_regs, rip = some kernel address. This is NOT what perf reads.

2. Before hrtimer Fires — No pt_regs Exists

foo() is running normally inside the child process (PID 4261). The CPU rip register updates every nanosecond:

```
foo() executing inside recursive process (PID 4261)

CPU rip register changes every nanosecond:
  t=0ns:    rip = 0x56068ffbc129  <- foo entry (endbr64)
  t=1ns:    rip = 0x56068ffbc12d  <- push %rbp
  t=2ns:    rip = 0x56068ffbc12e  <- mov %rsp,%rbp
  t=3ns:    rip = 0x56068ffbc131  <- sub $0x10,%rsp
  t=4ns:    rip = 0x56068ffbc135  <- mov %edi,-0x4(%rbp)
  ...billions of changes...
  t=10ms:   rip = 0x56068ffbc14e  <- hrtimer fires HERE

At this exact moment:
  pt_regs   = DOES NOT EXIST   <- nothing stored anywhere yet
  ring buffer = EMPTY
  IP exists ONLY inside CPU rip register
```

> ⚠ **NOTE: pt_regs is created AT the moment of interrupt. It does not exist before. It is destroyed after perf reads it. Its lifetime is microseconds.**

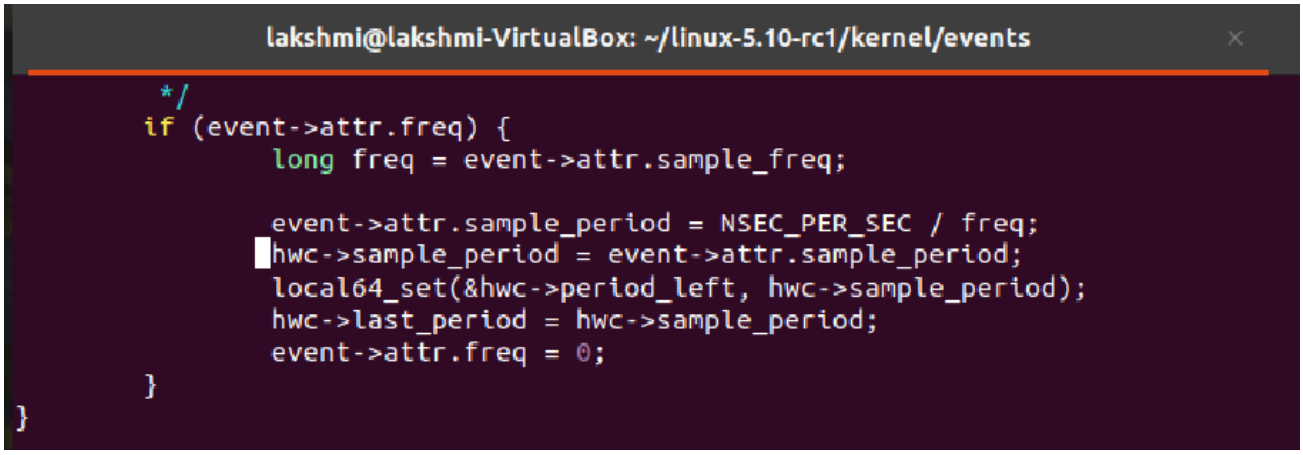
3. Step 1 — APIC Timer Chip Fires

The APIC (Advanced Programmable Interrupt Controller) is a hardware chip. mysampler set up the hrtimer with period = 10,000,000 ns. When it expires:

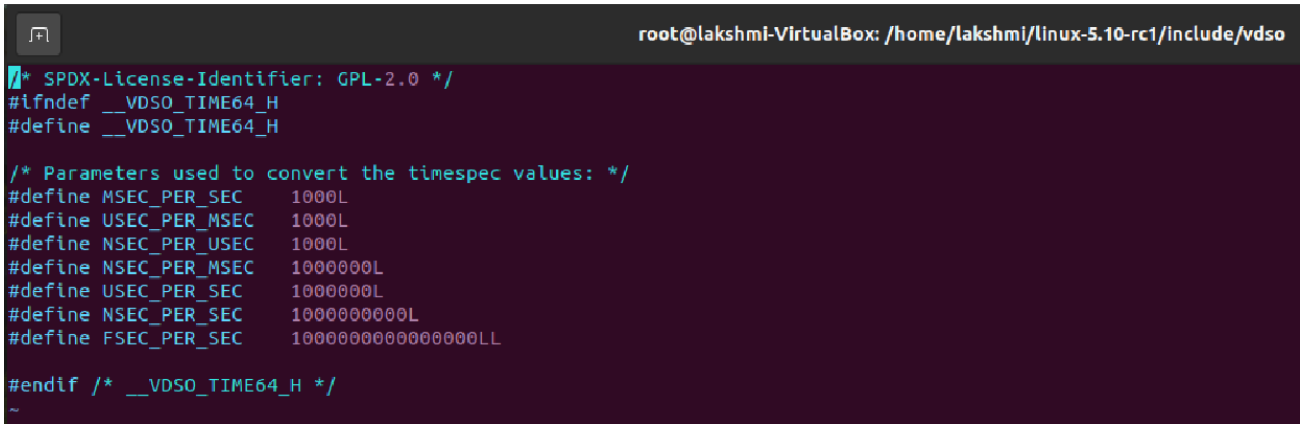
```
perf_swevent_init_hrtimer() registered:
  hwc->hrtimer.function = perf_swevent_hrtimer
  hwc->sample_period    = NSEC_PER_SEC / 100
                        = 1,000,000,000 / 100 = 10,000,000 ns

After ioctl(PERF_EVENT_IOC_ENABLE):
  hrtimer_start(&hwc->hrtimer, ns_to_ktime(10,000,000))
  <- hrtimer ARMED on child CPU context PID 4261

t = 10ms exactly:
  APIC timer chip sends ELECTRICAL SIGNAL to CPU pin
  This is purely hardware — no software caused this
```



Picture-1:-- perf_swevent_init_hrtimer() in kernel/events/core.c showing sample_period = NSEC_PER_SEC / freq computation.



Picture-2:-- include/vdso/time64.h showing NSEC_PER_SEC = 1000000000L and all time unit constants

4. Step 2 — CPU Finishes Current Instruction

The x86 specification says: complete the current instruction before handling an interrupt. The CPU obeys this rule in hardware:

```
foo() was executing: sub $0x1,%eax  at 0x56068ffbc14e

CPU rule (Intel x86 spec):
  'Finish current instruction first, then handle interrupt'

CPU finishes: sub $0x1,%eax
rip now advances to next instruction = 0x56068ffbc151

THIS 0x56068ffbc151 is the IP that gets captured.
It is the NEXT instruction foo() would have executed.
```

5. Step 3 — CPU Hardware Saves rip ATOMICALLY

This is the CAPTURE MOMENT. No software instruction. No kernel code. The CPU chip does this in silicon in one uninterruptible atomic sequence:

```
CPU HARDWARE — IN SILICON — no software instruction involved:

  1. reads rip register = 0x56068ffbc14e
     from CPU register (inside the CPU chip)

  2. reads TSS (Task State Segment)
     TSS is a hardware structure kernel set up at boot
     contains pointer to kernel stack
     CPU switches rsp to kernel stack

  3. pushes 5 values to kernel stack IN ONE ATOMIC SEQUENCE:
     ┌──────────────────────────────────────────────────┐
     │ pushes ss     = 0x2b   (userspace stack segment)│
     │ pushes rsp    = 0x7fff5678  (userspace rsp)     │
     │ pushes rflags = 0x246    (CPU flags)            │
     │ pushes cs     = 0x33   (userspace code segment) │
     │ pushes rip    = 0x56068ffbc14e  <- THE CAPTURE  │
     └──────────────────────────────────────────────────┘

  4. jumps to asm_common_interrupt in entry_64.S
     rip now = kernel code address
     foo's IP is ALREADY saved on kernel stack
     it CANNOT be lost or changed now
```

⚠ **NOTE: The Intel x86 manual guarantees steps 2 and 3 are atomic. No instruction executes between saving rip and jumping to the kernel. This is why the captured IP is always correct.**

5.1 Why rip does not point to kernel when captured

A common question: when kernel code runs, doesn't rip point to kernel addresses? Yes — but that does not affect the captured value:

```
WRONG mental model:
  kernel runs first -> kernel reads its own rip
  rip = kernel address -> WRONG IP captured

CORRECT model:
  hardware saves rip to MEMORY first     <- step 3 above
  THEN rip changes to kernel address     <- step 4 above
  no instruction executes between these  <- atomic

Two different things at the same time:

  CPU rip REGISTER:                KERNEL STACK MEMORY:
  0xffffffff8112xxxx               pt_regs->ip = 0x56068ffbc14e
  (kernel code, changes always)    (saved by hardware, frozen)

get_irq_regs() reads from MEMORY   <- not from rip register
regs->ip = 0x56068ffbc14e          <- CORRECT foo address
```

6. Step 4 — Software Completes pt_regs

Hardware saved 5 fields. Software (entry_64.S) now pushes the remaining 16 fields to complete the struct pt_regs layout:

6.1 idtentry macro — entry_64.S

```
FILE: arch/x86/entry/entry_64.S

.macro idtentry vector asmsym cfunc has_error_code:req
SYM_CODE_START(\asmsym)         <- CPU jumped HERE
    UNWIND_HINT_IRET_REGS
    ASM_CLAC
    .if \has_error_code == 0
        pushq   $-1             <- pt_regs->orig_ax = -1
    .endif
    idtentry_body \cfunc \has_error_code
SYM_CODE_END(\asmsym)
.endm
```

6.2 idtentry_body calls error_entry — entry_64.S

```
FILE: arch/x86/entry/entry_64.S

.macro idtentry_body cfunc has_error_code:req
    call    error_entry         <- calls error_entry
    UNWIND_HINT_REGS
    movq    %rsp, %rdi          <- pt_regs pointer -> rdi
    call    \cfunc              <- calls do_IRQ
    jmp     error_return
.endm
```

6.3 error_entry calls PUSH_AND_CLEAR_REGS — entry_64.S

```
FILE: arch/x86/entry/entry_64.S

SYM_CODE_START_LOCAL(error_entry)
    UNWIND_HINT_FUNC
    cld
    PUSH_AND_CLEAR_REGS save_ret=1  <- fills remaining 15 fields
    ENCODE_FRAME_POINTER 8
    testb   $3, CS+8(%rsp)
    jz      .Lerror_kernelspace
    SWAPGS
    ...
    ret
SYM_CODE_END(error_entry)
```

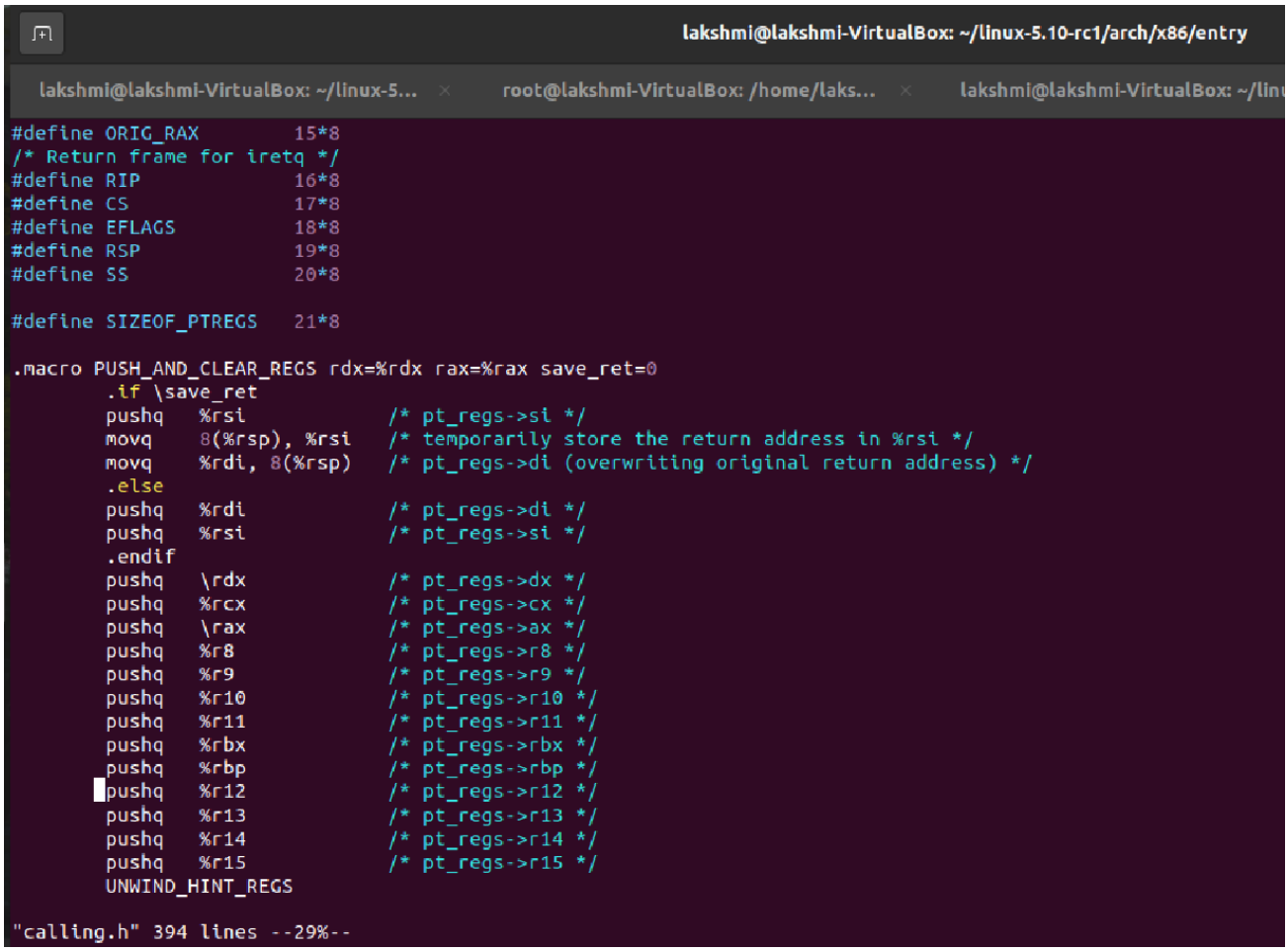
6.4 PUSH_AND_CLEAR_REGS — calling.h

```
FILE: arch/x86/entry/calling.h

.macro PUSH_AND_CLEAR_REGS rdx=%rdx rax=%rax save_ret=0
    .if \save_ret
        pushq   %rsi            /* pt_regs->si  offset 104 */
        movq    8(%rsp), %rsi   /* save return address     */
        movq    %rdi, 8(%rsp)   /* pt_regs->di  offset 112 */
    .endif
    pushq   \rdx               /* pt_regs->dx  offset 96  */
    pushq   %rcx               /* pt_regs->cx  offset 88  */
    pushq   \rax               /* pt_regs->ax  offset 80  */
    pushq   %r8                /* pt_regs->r8  offset 72  */
    pushq   %r9                /* pt_regs->r9  offset 64  */
    pushq   %r10               /* pt_regs->r10 offset 56  */
    pushq   %r11               /* pt_regs->r11 offset 48  */
    pushq   %rbx               /* pt_regs->bx  offset 40  */
    pushq   %rbp               /* pt_regs->bp  offset 32  */
    pushq   %r12               /* pt_regs->r12 offset 24  */
    pushq   %r13               /* pt_regs->r13 offset 16  */
    pushq   %r14               /* pt_regs->r14 offset 8   */
    pushq   %r15               /* pt_regs->r15 offset 0   */
    /* Zero registers for Spectre protection: */
    xorl    %edx,  %edx        /* nospec dx   */
    xorl    %ecx,  %ecx        /* nospec cx   */
```

```
    xorl    %r8d,  %r8d         /* nospec r8   */
    xorl    %r9d,  %r9d         /* nospec r9   */
    xorl    %r10d, %r10d        /* nospec r10  */
    xorl    %r11d, %r11d        /* nospec r11  */
    xorl    %ebx,  %ebx         /* nospec rbx  */
    xorl    %ebp,  %ebp         /* nospec rbp  */
    xorl    %r12d, %r12d        /* nospec r12  */
    xorl    %r13d, %r13d        /* nospec r13  */
    xorl    %r14d, %r14d        /* nospec r14  */
    xorl    %r15d, %r15d        /* nospec r15  */
.endm
```

⚠ **NOTE: PUSH_AND_CLEAR_REGS is a MACRO not a function. The assembler copies its instructions inline. No call/ret. No stack frame. The 15 pushq instructions directly extend the pt_regs struct on the kernel stack.**



Picture-3:-- calling.h showing complete PUSH_AND_CLEAR_REGS macro with all pushq instructions and SIZEOF_PTREGS = 21*8.

7. Complete pt_regs Layout After PUSH_AND_CLEAR_REGS

After all pushes, the kernel stack contains a complete struct pt_regs (21 fields x 8 bytes = 168 bytes):

Offset	Field and Value	Who Stored It
0	r15 = 0 (zeroed)	PUSH_AND_CLEAR_REGS (calling.h)
8	r14 = 0 (zeroed)	PUSH_AND_CLEAR_REGS (calling.h)
16	r13 = 0 (zeroed)	PUSH_AND_CLEAR_REGS (calling.h)
24	r12 = 0 (zeroed)	PUSH_AND_CLEAR_REGS (calling.h)
32	rbp = 0 (zeroed)	PUSH_AND_CLEAR_REGS (calling.h)
40	rbx = 0 (zeroed)	PUSH_AND_CLEAR_REGS (calling.h)
48	r11 = 0 (zeroed)	PUSH_AND_CLEAR_REGS (calling.h)
56	r10 = 0 (zeroed)	PUSH_AND_CLEAR_REGS (calling.h)
64	r9 = 0 (zeroed)	PUSH_AND_CLEAR_REGS (calling.h)
72	r8 = 0 (zeroed)	PUSH_AND_CLEAR_REGS (calling.h)
80	rax = foo's rax value	PUSH_AND_CLEAR_REGS (calling.h)
88	rcx = 0 (zeroed)	PUSH_AND_CLEAR_REGS (calling.h)
96	rdx = 0 (zeroed)	PUSH_AND_CLEAR_REGS (calling.h)
104	rsi = foo's rsi value	PUSH_AND_CLEAR_REGS (calling.h)
112	rdi = foo's rdi value	PUSH_AND_CLEAR_REGS (calling.h)
120	orig_ax = -1	idtentry macro (entry_64.S)
128	ip = 0x56068ffbc14e THE SAMPLE	CPU HARDWARE (no software instruction)
136	cs = 0x33	CPU HARDWARE (no software instruction)
144	flags = 0x246	CPU HARDWARE (no software instruction)
152	sp = 0x7fff5678	CPU HARDWARE (no software instruction)
160	ss = 0x2b	CPU HARDWARE (no software instruction)

8. Step 5 — Kernel Reads pt_regs->ip

8.1 do_IRQ stores pt_regs pointer

```
FILE: arch/x86/kernel/irq.c

__visible void __irq_entry do_IRQ(struct pt_regs *regs)
{
    /* regs = rsp = start of pt_regs on kernel stack */
    struct pt_regs *old_regs = set_irq_regs(regs);
    /* FILE: arch/x86/include/asm/irq_regs.h           */
    /* stores pt_regs pointer in __irq_regs per-CPU var*/
    /* so perf_swevent_hrtimer can find it later       */
    ...
}
```

8.2 set_irq_regs saves the pointer

```
FILE: arch/x86/include/asm/irq_regs.h

static inline struct pt_regs *
set_irq_regs(struct pt_regs *new_regs)
{
    struct pt_regs *old_regs;
    old_regs = __this_cpu_read(__irq_regs);
    __this_cpu_write(__irq_regs, new_regs);
    /* __irq_regs now points to pt_regs on kernel stack */
    /* pt_regs->ip = 0x56068ffbc14e                     */
    return old_regs;  /* for nested interrupt handling  */
}
```

8.3 perf_swevent_hrtimer reads pt_regs

```
FILE: kernel/events/core.c  line 10272

static enum hrtimer_restart
perf_swevent_hrtimer(struct hrtimer *hrtimer)
{
    struct perf_sample_data data;
    struct pt_regs *regs;

    event = container_of(hrtimer, struct perf_event, hw.hrtimer);

    if (event->state != PERF_EVENT_STATE_ACTIVE)
        return HRTIMER_NORESTART;

    perf_sample_data_init(&data, 0, event->hw.last_period);

    regs = get_irq_regs();
    /* FILE: arch/x86/include/asm/irq_regs.h          */
    /* reads __irq_regs per-CPU variable              */
    /* returns pointer to pt_regs on kernel stack     */
```

```
    /* regs->ip = 0x56068ffbc14e  <- foo IP            */

    if (regs && !perf_exclude_event(event, regs))
        __perf_event_overflow(event, 1, &data, regs);
    /* passes regs to overflow handler                 */
    /* which reads regs->ip and writes to ring buffer  */

    hrtimer_forward_now(hrtimer, ns_to_ktime(period));
    return HRTIMER_RESTART;  /* fire again in 10ms    */
}
```

8.4 get_irq_regs returns the pointer

```
FILE: arch/x86/include/asm/irq_regs.h

static inline struct pt_regs *get_irq_regs(void)
{
    return this_cpu_read(__irq_regs);
    /* reads __irq_regs per-CPU variable               */
    /* returns pointer to pt_regs on kernel stack      */
    /* NO copy — just a pointer to existing memory     */
    /* regs->ip is at memory[rsp + 128]                */
    /*         = 0x56068ffbc14e                        */
}
```

9. Step 6 — Reading pt_regs->ip and Writing to Ring Buffer

9.1 perf_prepare_sample reads regs->ip

```
FILE: kernel/events/core.c

void perf_prepare_sample(struct perf_event_header *header,
                         struct perf_sample_data *data,
                         struct perf_event *event,
                         struct pt_regs *regs)
{
    u64 sample_type = event->attr.sample_type;

    header->type = PERF_RECORD_SAMPLE;
    header->misc |= perf_misc_flags(regs);
    /* reads regs->cs = 0x33 = userspace -> PERF_RECORD_MISC_USER */

    if (sample_type & PERF_SAMPLE_IP)       /* TRUE for mysampler */
        data->ip = perf_instruction_pointer(regs);
        /* THIS LINE reads pt_regs->ip                */
        /* = regs->ip + code_segment_base(regs)       */
        /* = 0x56068ffbc14e + 0   (64-bit mode)       */
        /* = 0x56068ffbc14e       <- foo IP captured  */
}
```

9.2 perf_instruction_pointer — reads regs->ip

```
FILE: arch/x86/events/core.c

unsigned long perf_instruction_pointer(struct pt_regs *regs)
{
        if (perf_guest_cbs && perf_guest_cbs->is_in_guest())
                return perf_guest_cbs->get_guest_ip();

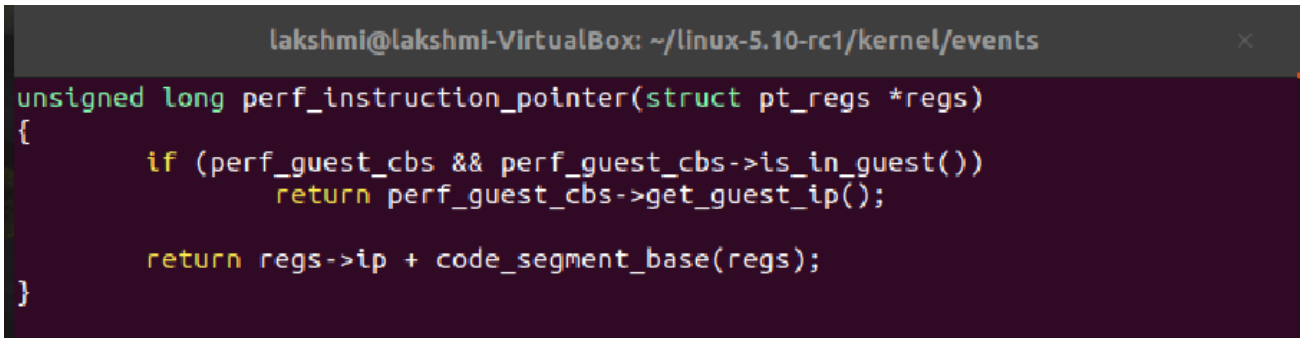
        return regs->ip + code_segment_base(regs);
}
    /* code_segment_base() returns 0 for 64-bit mode   */
    /* cs = 0x33 -> user_64bit_mode = TRUE -> return 0 */
    /* result = regs->ip + 0 = 0x56068ffbc14e          */
}
```

9.3 code_segment_base returns 0 for 64-bit

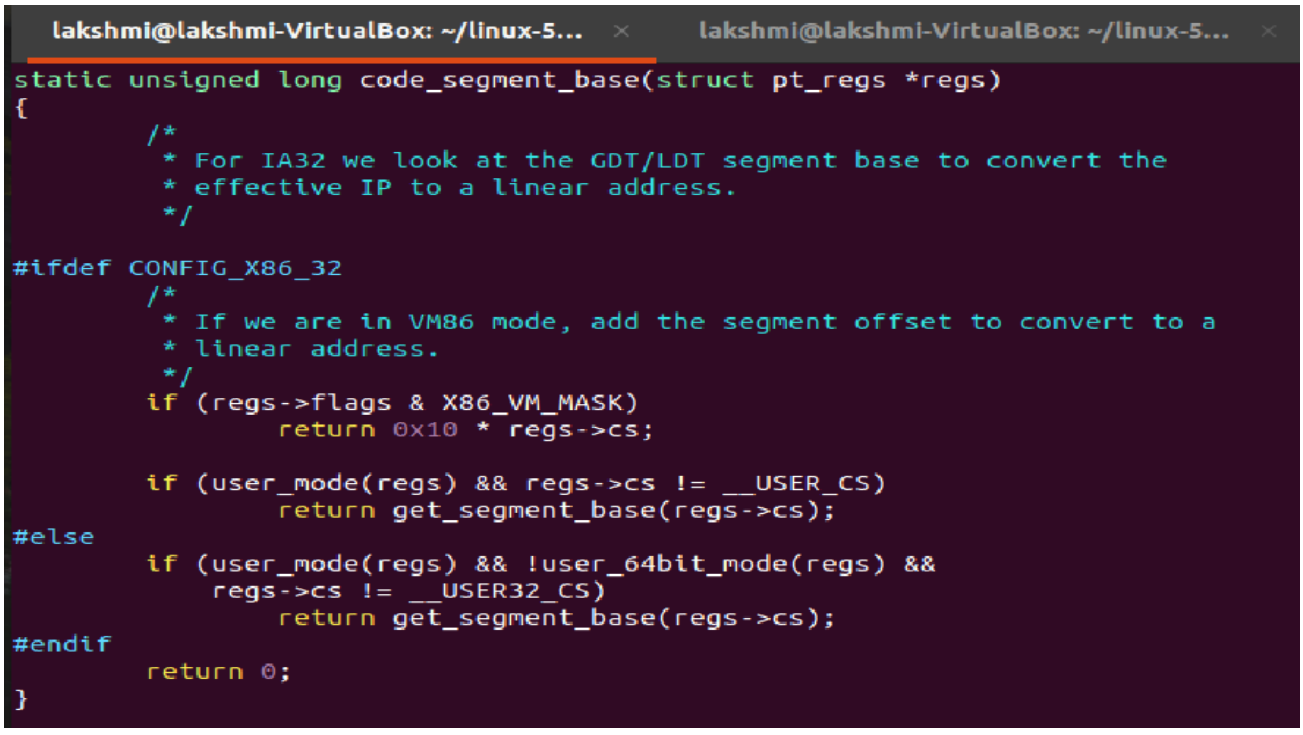
```
FILE: arch/x86/kernel/perf_event.c

static unsigned long code_segment_base(struct pt_regs *regs)
{
#ifdef CONFIG_X86_32
    if (regs->flags & X86_VM_MASK)   /* VM86 16-bit mode */
        return 0x10 * regs->cs;
```

```
    if (user_mode(regs) && regs->cs != __USER_CS)
        return get_segment_base(regs->cs);
#else
    if (user_mode(regs) && !user_64bit_mode(regs) &&
        regs->cs != __USER32_CS)
        return get_segment_base(regs->cs);
#endif
    return 0;
    /* OUR CASE: cs = 0x33 = 64-bit userspace           */
    /* user_64bit_mode = TRUE -> returns 0              */
    /* ip + 0 = ip = 0x56068ffbc14e                     */
}
```



Picture-4:-- arch/x86/events/core.c showing perf_instruction_pointer() returning regs->ip + code_segment_base(regs).



Picture-4:-- arch/x86/kernel/perf_event.c showing code_segment_base() returning 0 for 64-bit mode (cs=0x33).

10. Step 7 — Writing IP to Ring Buffer

10.1 perf_output_sample writes data->ip

```
FILE: kernel/events/core.c

void perf_output_sample(struct perf_output_handle *handle,
                        struct perf_event_header *header,
                        struct perf_sample_data *data,
                        struct perf_event *event)
{
    u64 sample_type = data->type;

    perf_output_put(handle, *header);
    /* writes 8 bytes: [type=PERF_RECORD_SAMPLE][misc][size=48] */

    if (sample_type & PERF_SAMPLE_IP)       /* TRUE for mysampler */
        perf_output_put(handle, data->ip);
        /* writes 8 bytes: 0x56068ffbc14e to ring buffer */
        /* expands to: __output_copy(handle, &data->ip, 8) */

    if (sample_type & PERF_SAMPLE_TID)    perf_output_put(handle, data->tid_entry);
    if (sample_type & PERF_SAMPLE_TIME)   perf_output_put(handle, data->time);
    if (sample_type & PERF_SAMPLE_PERIOD) perf_output_put(handle, data->period);
    /* total: 8 header + 8+8+8+8 = 48 bytes per record */
}
```

10.2 __DEFINE_OUTPUT_COPY_BODY — lowest level memcpy

```
FILE: kernel/events/internal.h

#define __DEFINE_OUTPUT_COPY_BODY(advance_buf, memcpy_func, ...)
{
    unsigned long size, written;
    do {
        size    = min(handle->size, len);     /* = 8 bytes */
        written = memcpy_func(__VA_ARGS__);
        /* memcpy(handle->addr, &data->ip, 8)            */
        /* copies 0x56068ffbc14e to ring buffer page     */
        written = size - written;             /* = 8     */
        len    -= written;                    /* = 0     */
        handle->addr += written;   /* advance write ptr  */
        handle->size -= written;
        if (!handle->size) {        /* page full? */
            handle->page++;
            handle->page &= rb->nr_pages - 1; /* circular wrap */
            handle->addr = rb->data_pages[handle->page];
            handle->size = PAGE_SIZE << page_order(rb);
        }
    } while (len && written == size);
    return len;   /* 0 = all bytes written successfully */
}
```

10.3 Ring buffer after write

```
shared memory (pages 1-128) after first hrtimer fire:

offset 0:   [type=PERF_RECORD_SAMPLE][misc][size=48]  <- 8 bytes header
offset 8:   [0x56068ffbc14e                        ]  <- 8 bytes IP
offset 16:  [pid=4261            ][tid=4261        ]  <- 8 bytes TID
offset 24:  [timestamp                             ]  <- 8 bytes TIME
offset 32:  [10000000                              ]  <- 8 bytes PERIOD
offset 40:  ... (next record starts here)

meta->data_head = 48  <- kernel advanced by 48 bytes
meta->data_tail = 0   <- mysampler has not read yet
```

lakshmi@lakshmi-VirtualBox: ~/linux-5.10-rc1

```
#define __DEFINE_OUTPUT_COPY_BODY(advance_buf, memcpy_func, ...)        \
{                                                                       \
        unsigned long size, written;                                    \
                                                                        \
        do {                                                            \
                size    = min(handle->size, len);                       \
                written = memcpy_func(__VA_ARGS__);                     \
                written = size - written;                               \
                                                                        \
                len -= written;                                         \
                handle->addr += written;                                \
                if (advance_buf)                                        \
                        buf += written;                                 \
                handle->size -= written;                                \
                if (!handle->size) {                                    \
                        struct perf_buffer *rb = handle->rb;    \
                                                                        \
                        handle->page++;                                 \
                        handle->page &= rb->nr_pages - 1;               \
                        handle->addr = rb->data_pages[handle->page];    \
                        handle->size = PAGE_SIZE << page_order(rb);     \
                }                                                       \
        } while (len && written == size);                               \
                                                                        \
        return len;                                                     \
}
```

Picture-4:-- kernel/events/internal.h showing __DEFINE_OUTPUT_COPY_BODY macro with memcpy loop that writes IP to ring buffer pages.

11. Step 8 — mysampler Reads the IP

```
FILE: mysampler.c
FUNCTION: read_ring_buffer()

uint64_t head = __atomic_load_n(&meta->data_head, __ATOMIC_ACQUIRE);
/* head = 48 (one sample written by kernel) */

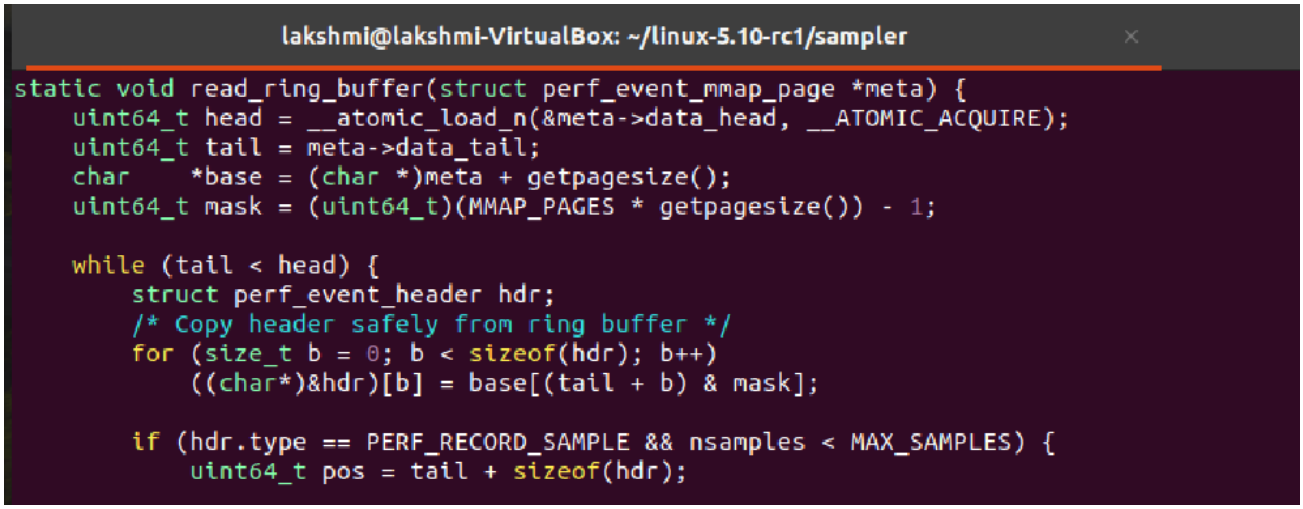
char    *base = (char *)meta + getpagesize();
/* skip metadata page, point to ring buffer data */

uint64_t mask = (uint64_t)(MMAP_PAGES * getpagesize()) - 1;
/* mask = 0x7FFFF for circular wrap-around */

/* reads 8-byte header at tail=0 */
hdr.type = PERF_RECORD_SAMPLE  <- confirmed
hdr.size = 48

uint64_t pos = tail + sizeof(hdr);  /* = 0 + 8 = 8 */
for (size_t b = 0; b < sizeof(ip); b++)
    ((char*)&ip)[b] = base[(pos + b) & mask];
/* reads 8 bytes from ring buffer */
/* ip = 0x56068ffbc14e  <- THE CAPTURED IP  */

samples[nsamples++] = ip;
/* samples[0] = 0x56068ffbc14e */
```



Picture-4:-- mysampler.c showing read_ring_buffer() function reading raw IP addresses from shared memory ring buffer.

12. Complete Chain — All Steps with File Locations

Step	What Happens	File
1 foo() running	rip = 0x56068ffbc14e in CPU register only	NO FILE
2 APIC signal	Hardware timer chip sends signal to CPU pin	NO FILE — hardware chip
3 CPU finishes instr	sub $0x1,%eax completes at 0x56068ffbc14e	NO FILE — x86 spec
4 CPU saves rip	HARDWARE pushes rip=0x56068ffbc14e to kernel stack ATOMICALLY	NO FILE — CPU silicon
5 CPU pushes cs,flags...	HARDWARE pushes cs,rflags,rsp,ss to stack	NO FILE — CPU silicon
6 CPU jumps to handler	rip = asm_common_interrupt. foo IP already saved.	IDT — set at boot
7 idtentry macro	pushq $-1 (orig_ax). calls idtentry_body	arch/x86/entry/entry_64.S
8 idtentry_body macro	call error_entry	arch/x86/entry/entry_64.S
9 error_entry()	PUSH_AND_CLEAR_REGS save_ret=1	arch/x86/entry/entry_64.S
10 PUSH_AND_CLEAR_REGS	pushq %rdi..%r15. pt_regs COMPLETE.	arch/x86/entry/calling.h
11 movq %rsp, %rdi	passes pt_regs pointer as argument to do_IRQ	arch/x86/entry/entry_64.S
12 do_IRQ()	set_irq_regs(regs) stores pointer in __irq_regs	arch/x86/kernel/irq.c
13 set_irq_regs()	__this_cpu_write(__irq_regs, regs)	arch/x86/include/asm/irq_regs.h
14 perf_swevent_hrtimer()	regs = get_irq_regs(). calls __perf_event_overflow()	kernel/events/core.c:10272
15 get_irq_regs()	return this_cpu_read(__irq_regs) -> pt_regs*	arch/x86/include/asm/irq_regs.h
16 perf_exclude_event()	cs=0x33 = userspace -> PASS. exclude_kernel=1 check.	kernel/events/core.c
17 __perf_event_overflow()	READ_ONCE(event->overflow_handler)(event,data,regs)	kernel/events/core.c:8874
18 perf_event_output_forward()	calls __perf_event_output()	kernel/events/core.c
19 perf_prepare_sample()	data->ip = perf_instruction_pointer(regs)	kernel/events/core.c

20 perf_instruction_pointer()	return regs->ip + code_segment_base(regs)	arch/x86/include/asm/perf_event.h
21 code_segment_base()	cs=0x33 -> user_64bit_mode=TRUE -> return 0	arch/x86/kernel/perf_event.c
22 perf_output_sample()	perf_output_put(handle, data->ip) -> __output_copy	kernel/events/core.c
23 __DEFINE_OUTPUT_COPY_BODY	memcpy(ring_buffer, &data->ip, 8) -> IP written	kernel/events/internal.h
24 data_head advances	perf_output_end() -> data_head += 48	kernel/events/core.c
25 read_ring_buffer()	ip = base[(pos) & mask] = 0x56068ffbc14e	mysampler.c
26 samples[] stored	samples[nsamples++] = 0x56068ffbc14e	mysampler.c
27 va_to_elf()	elf_va = ip - load_bias = 0x114e	mysampler.c
28 find_sym()	0x1129 <= 0x114e < 0x115d -> 'foo'	mysampler.c
29 foo = 100%	print_histogram() -> foo holds all CPU time	mysampler.c

13. Syscall Path vs hrtimer Path

There are two ways pt_regs gets filled. Only the hrtimer path captures foo's IP:

pt_regs field	Syscall path (entry_SYSCALL_64)	hrtimer path (our case)
`ip`	pushq %rcx (SOFTWARE) = mysampler return addr. NOT a sample IP.	CPU HARDWARE pushes rip = foo's IP. THE SAMPLE.
`cs`	pushq $__USER_CS (SOFTWARE constant)	CPU HARDWARE pushes cs register
`flags`	pushq %r11 (SOFTWARE — syscall saved rflags here)	CPU HARDWARE pushes rflags register
`sp`	pushq saved rsp from TSS_sp2 (SOFTWARE)	CPU HARDWARE pushes rsp register
`ss`	pushq $__USER_DS (SOFTWARE constant)	CPU HARDWARE pushes ss register
`orig_ax`	pushq %rax = syscall number 298 (SOFTWARE)	pushq $-1 by idtentry macro (SOFTWARE)
`r15-rdi`	PUSH_AND_CLEAR_REGS (SOFTWARE)	PUSH_AND_CLEAR_REGS (SOFTWARE — same macro)
`Sample?`	NO. Only mysampler's return address.	YES. foo's exact instruction address.

14. Summary in Five Lines

- ✔ **APIC timer chip fires electrical signal to CPU every 10ms (hardware).**
- ✔ **CPU hardware saves rip=0x56068ffbc14e to kernel stack BEFORE any kernel code runs (atomic, in silicon).**
- ✔ **Software (PUSH_AND_CLEAR_REGS in calling.h) completes pt_regs struct around the saved rip.**
- ✔ **perf_swevent_hrtimer reads regs->ip from kernel stack memory via get_irq_regs().**
- ✔ **memcpy writes raw IP to ring buffer — mysampler reads it, resolves via ASLR reversal to 'foo = 100%'.**